

ServiceNow GlideRecord CRUD Operations: Detailed Reference

`GlideRecord` is the primary server-side class in ServiceNow used for database operations. It allows developers to perform standard CRUD (Create, Read, Update, Delete) operations on ServiceNow tables.

This guide provides detailed explanations, use cases, and code templates for each database operation.

1. READ Operations (Retrieve Data)

Read operations are used to query the database and fetch records based on specific filters.

A. Retrieve a Single Record via `get()`

If you know the unique `sys\_id` or a specific unique field value (like ticket number), `get()` is the fastest and most efficient way to query a single record.

- Why Used: Bypasses the need for `addQuery()` and `query()`, performing a direct single-row fetch.
- Code Example:

```
var gr = new GlideRecord('incident');
// Retrieve record directly by Sys ID
if (gr.get('sys_id', '9d3b0e3ec611227c01d2deec05531768')) {
    gs.info('Incident Number: ' + gr.getValue('number'));
}

// Or retrieve directly by unique Number field
var gr2 = new GlideRecord('incident');
if (gr2.get('number', 'INC0009001')) {
    gs.info('Short Description: ' + gr2.getValue('short_description'));
}
```

B. Query Multiple Records via `addQuery()` & `query()`

For querying multiple records, chain filters using `addQuery()` and load them with `query()`. Loop through results using `next()`.

- Why Used: To fetch lists of records matching specific criteria (e.g., active incidents in a group).
- Code Example:

```
var gr = new GlideRecord('incident');
gr.addQuery('active', true);
gr.addQuery('priority', '1'); // 1 = Critical
gr.query();

// Loop through each matching record
while (gr.next()) {
    gs.info('Critical Ticket: ' + gr.getValue('number') + ' - ' + gr.getValue('short_description'));
}
```

C. Encoded Query Filters via `addEncodedQuery()`

An encoded query is a single query string representing multiple conditions (similar to breadcrumbs on lists).

- Why Used: Simplifies complex filter criteria into a single copy-pasteable string from the list filter UI.
- Code Example:

```
var gr = new GlideRecord('incident');
// Active is true, Category is software, Priority is 1 or 2
var filterString = "active=true^category=software^priorityIN1,2";
gr.addEncodedQuery(filterString);
gr.query();

while (gr.next()) {
```

```
gs.info('Found Ticket: ' + gr.number);
}
```

2. CREATE Operations (Insert Data)

Create operations instantiate a new record in memory, populate field values, and commit it to the database.

A. Single Record Insertion via `insert()`

- Why Used: To create new records automatically (e.g., creating an emergency task when an incident is logged).
- Code Example:

```
var gr = new GlideRecord('incident');
// 1. Initialize a new blank record structure in memory
gr.initialize();

// 2. Set field values
gr.setValue('short_description', 'Critical database outage reported');
gr.setValue('category', 'database');
gr.setValue('urgency', '1');
gr.setValue('impact', '1');

// 3. Commit the record to the database
var newSysId = gr.insert();
gs.info('Successfully created new Incident Sys ID: ' + newSysId);
```

`initialize()` vs `newRecord()`:

** `initialize()` creates a blank record in memory but does not apply default values or generate a Sys ID until `insert()` is called.*

** `newRecord()` instantiates the record, applies all dictionary default values, and automatically generates a unique Sys ID in memory before `insert()` is executed (recommended when reference child tasks require the parent Sys ID first).*

3. UPDATE Operations (Modify Data)

Update operations query existing records in the database, modify their field properties, and save changes.

A. Updating a Single Record via `update()`

- Why Used: To modify specific properties of a single ticket (e.g., resolving an incident).
- Code Example:

```
var gr = new GlideRecord('incident');
if (gr.get('number', 'INC0009001')) {
    // Modify target fields
    gr.setValue('state', '6'); // 6 = Resolved
    gr.setValue('close_code', 'Solved (Permanently)');
    gr.setValue('close_notes', 'Resolved via script update.');

    // Save modifications to the database
    gr.update();
    gs.info('Incident updated successfully.');
```

B. Bulk Updates via `updateMultiple()`

- Why Used: To update hundreds or thousands of records matching a query in one database roundtrip. Never loop `update()` inside a `while (next())` loop, as it generates high network query overhead.
- Code Example:

```
var gr = new GlideRecord('incident');
```

```
gr.addQuery('active', true);
gr.addQuery('state', '3'); // 3 = On Hold
gr.addQuery('hold_reason', '4'); // 4 = Awaiting Vendor
gr.query();

// Set the target values to apply to all queried records
gr.setValue('work_notes', 'Bulk vendor status check update.');
```

```
gr.setValue('urgency', '3'); // Downgrade urgency

// Perform the bulk update
gr.updateMultiple();
gs.info('Bulk updates completed.');
```

4. DELETE Operations (Remove Data)

Delete operations permanently remove records from the database. Use with caution.

A. Deleting a Single Record via `deleteRecord()`

- Why Used: To remove a specific erroneous or duplicate record.
- Code Example:

```
var gr = new GlideRecord('sys_user');
if (gr.get('email', 'test.user@example.com')) {
  // Delete target record
  if (gr.deleteRecord()) {
    gs.info('Test user deleted successfully.');
```

```
  } else {
    gs.info('Failed to delete user.');
```

```
  }
}
```

B. Bulk Deletion via `deleteMultiple()`

- Why Used: Fast bulk cleanup of log files or temp files. Performs the delete in a single operation.
- Code Example:

```
var gr = new GlideRecord('u_temp_log_table');
gr.addQuery('sys_created_on', '<', gs.daysAgo(30)); // Older than 30 days
// Bulk delete all matched records
gr.deleteMultiple();
gs.info('Historical logs cleared.');
```

Crucial Utility Methods

- `gr.hasNext()`: Returns true if there is at least one record matching the query (checks without moving the cursor pointer).
- `gr.getRowCount()`: Returns the total number of records matching the query.
- `gr.setLimit(number)`: Restricts query results size. Essential when validating existence (e.g. `setLimit(1)`).
- `gr.orderBy(field)` / `gr.orderByDesc(field)`: Orders results ascending/descending.
- `gr.getValue(field)` / `gr.setValue(field, val)`: Safely reads/writes field data. Avoid direct dot-walking (like `gr.field = val`) to prevent object reference pointer bugs.

Server-Side Scripting Best Practices

1. No Client-Side GlideRecord: Never run `new GlideRecord()` in Client Scripts or UI Policies. It triggers synchronous queries that lock the browser. Use `GlideAjax` instead.

2. Limit Queries with ``setLimit()``: When checking if a record exists, use ``gr.setLimit(1)`` to stop the query engine from retrieving unnecessary rows.
3. Use ``updateMultiple()`` & ``deleteMultiple()`` for Bulk Operations: Do not loop ``gr.update()`` or ``gr.deleteRecord()`` inside a ``while(gr.next())`` loop. It causes performance bottlenecks due to excessive database requests.
4. Use Safe Getters/Setters: Always use ``gr.getValue('field')`` and ``gr.setValue('field', value)`` rather than direct assignments (``gr.field = value``) to ensure correct type coercion and prevent memory leak errors.
5. Use ``addNullQuery()`` and ``addNotNullQuery()``: When checking for empty or non-empty fields, use these built-in methods instead of checking against empty strings.